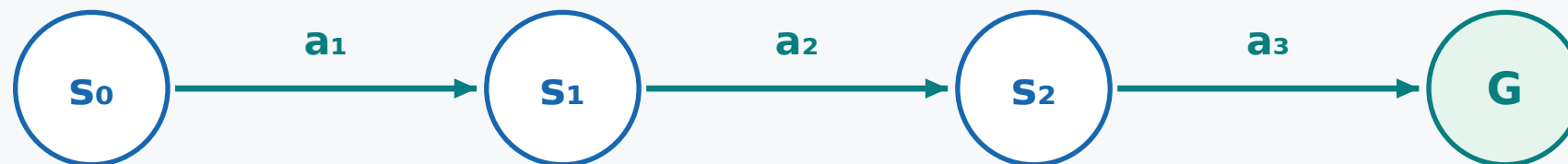


Resolver, ver y comprobar

Problemas de búsqueda explicados con estados, decisiones y evidencias.

**No basta con llegar.
Hay que explicar
por qué funciona.**



Material de apoyo para estudiantes · Sistemas Inteligentes
En español. Etiquetas y símbolos compatibles con los apuntes en inglés.

FORMULAR

Qué cambia y qué debe conservarse.

BUSCAR

Qué nodo sale de la frontera y por qué.

COMPROBAR

Validez, coste y posibilidad de solución.

Un recorrido por los ejercicios

Pulsa un bloque para abrirlo. Las preguntas de pausa preceden a la explicación.

01	Método y notación	3	09	Árbol numérico: objetivo 13	33
02	Cambio de monedas: 15 céntimos	6	10	Rumanía: A* desde Oradea	35
03	Misioneros y caníbales	9	11	Aspiradora y espacio de creencias	38
04	Lobo, oveja, col y barquero	14	12	Exploradores y capacidad del barco	40
05	Ping-pong: demostrar imposibilidad	17	13	8-puzzle: la meta de Unit2	42
06	DLS: el problema de la poda	19	14	Column Jump: ocho saltos	46
07	Grafo2: cinco estrategias	22	15	Best-first y búsqueda local	50
08	grafoclase: coste y reaperturas	29	16	Síntesis, fuentes y uso en clase	53

La misma plantilla para todos

Primero definimos el problema; después elegimos cómo recorrer sus posibilidades.

- 1 **ESTADO INICIAL**
¿Dónde empezamos?
- 2 **ACCIONES**
¿Qué movimientos son legales?
- 3 **TRANSICIÓN**
¿Qué estado produce cada acción?
- 4 **TEST DE META**
¿Cómo reconocemos que hemos terminado?
- 5 **COSTE**
¿Qué significa una solución mejor?

La función **Sucesor de los apuntes** reúne tres piezas.

Sucesor(s) devuelve (acción, estado resultante, coste): es compatible con ACTIONS, RESULT y c del libro.

Un estado no es un nodo

La misma configuración puede aparecer en rutas distintas.

ESTADO

Describe el mundo.

Ejemplo: Sibiu.

No contiene por sí mismo ni padre, ni profundidad, ni coste acumulado.

NODO DE BÚSQUEDA

Describe una llegada a ese estado.

id, estado, padre, acción, d, g, h, f.

Sibiu puede aparecer con $d=1$ y con $d=3$. Son el mismo estado y dos nodos diferentes.



$$d=1 \cdot g=140$$



$$d=3 \cdot g=297$$

Qué versión del algoritmo estamos trazando

Una convención explícita evita comparar números que no significan lo mismo.

Aspecto	En las trazas de los apuntes	En el BFS específico del libro
Prueba de meta	Al extraer el nodo	Al generar el sucesor
Repetidos	Se admiten en frontera; se comprueban al extraer	Se evitan en frontera y explorados
Profundidad	Raíz $d=0$; DLS admite la meta con $d=\ell$	Misma profundidad, distinta implementación
Prioridad	BFS: d · DLS: $-d$ · UCS: g	BFS: cola FIFO
Desempate de este dossier	Menor id de creación; sucesores ordenados	Se debe declarar en cada implementación

No confundir extraer, expandir y encontrar.

Un nodo CUT se extrae pero no se expande. La meta se extrae y se reconoce; no se generan sus hijos.

15 céntimos, con existencias limitadas

No buscamos divisas: buscamos una combinación de monedas.



1 moneda



4 monedas



1 moneda



4 monedas

ESTADO

$((n_{50}, n_{10}, n_5, n_1), X)$

n: monedas ya entregadas.

X: cantidad pendiente.

Inicio: $((0, 0, 0, 0), 15)$.

REGLA Y OBJETIVO

Entregar una moneda disponible que no supere X.

Actualizar contador y restar su valor.

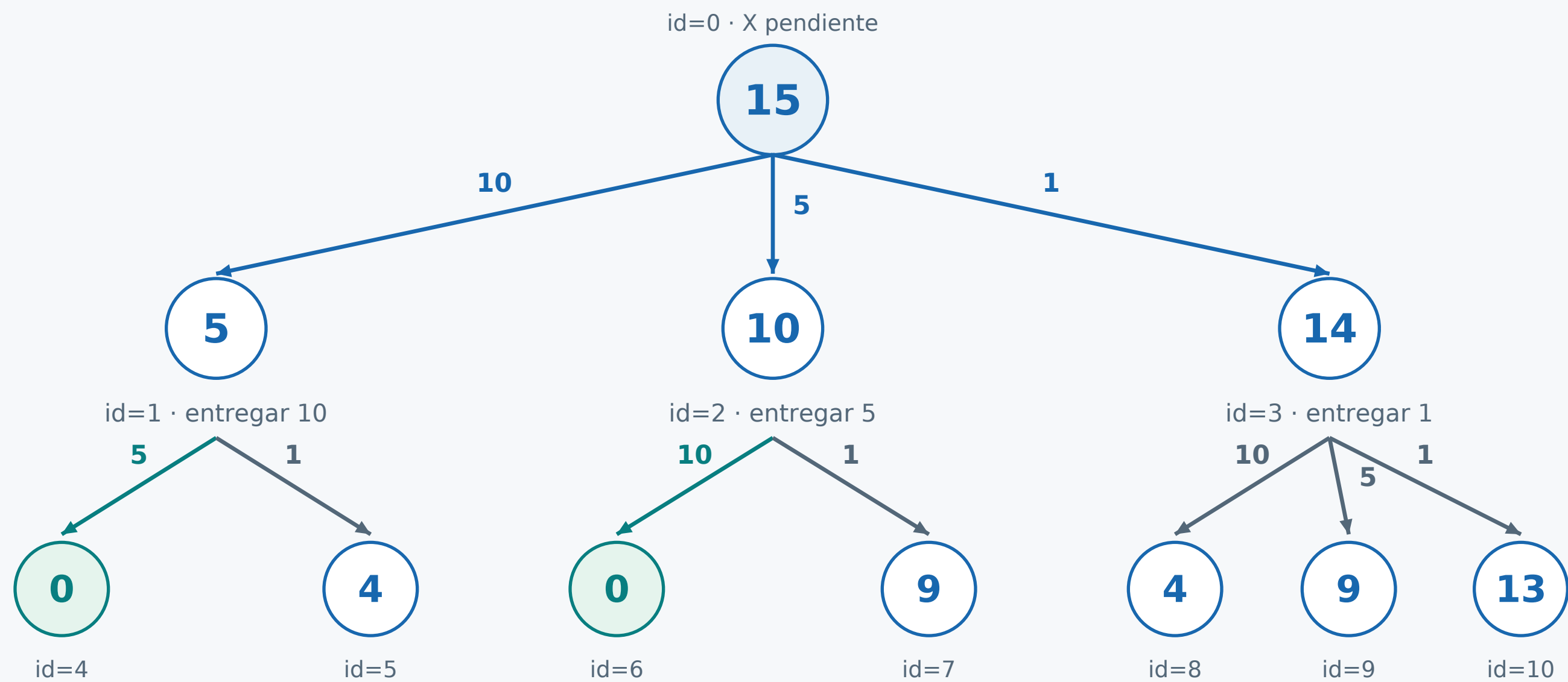
Meta: $X=0$. Coste: 1 por moneda.

Pausa antes del árbol

¿Por qué guardar solo X no describe, en general, un problema de cambio con existencias limitadas?

El árbol se abre por niveles

Orden de operadores del enunciado: 50, 10, 5, 1. El operador 50 no es aplicable.



Dos caminos, un mismo estado objetivo

10→5 y 5→10 llegan a ((0,1,1,0),0). Cambia el orden del camino, no la combinación final.

La frontera hace visible el orden

Los id son los del árbol anterior. La cabeza de la cola está a la izquierda.

Paso	Extraído	Frontera después	Decisión
0	—	[0]	Insertar la raíz
1	0: X=15	[1,2,3]	Generar 10, 5 y 1
2	1: X=5	[2,3,4,5]	Generar meta 4, pero no parar
3	2: X=10	[3,4,5,6,7]	Aparece otra llegada a la meta
4	3: X=14	[4,5,6,7,8,9,10]	Completar el nivel anterior
5	4: X=0	[5,6,7,8,9,10]	META · reconstruir 0→1→4

2

monedas: 10 + 5

2

acciones; coste óptimo

11

nodos creados, incluida la raíz

Representar sin duplicar información

Mantenemos las dos orillas del enunciado y añadimos una abreviatura equivalente.

NOTACIÓN ORIGINAL

$L \langle M \ C \ B \rangle \ R \langle M \ C \ B \rangle$

Inicio: $L \langle 3 \ 3 \ 1 \rangle \ R \langle 0 \ 0 \ 0 \rangle$.

Meta: $L \langle 0 \ 0 \ 0 \rangle \ R \langle 3 \ 3 \ 1 \rangle$.

ABREVIATURA DEL DOSSIER

(m, c, b) : cantidades a la izquierda y lado del barco, L o R.

A la derecha hay $3 - m$ misioneros y $3 - c$ caníbales.

00 INICIO

$(3, 3, L)$

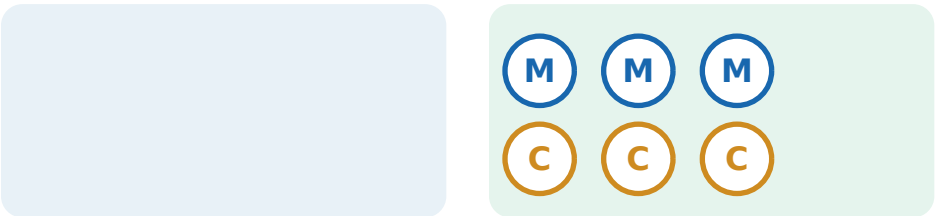


L [B]

R

11 META

$(0, 0, R)$



L

R [B]

Un movimiento debe ser seguro en ambas orillas

La hoja de formalización contiene solo un comienzo de la función sucesor.

Cargas candidatas: M, C, MM, CC y MC.

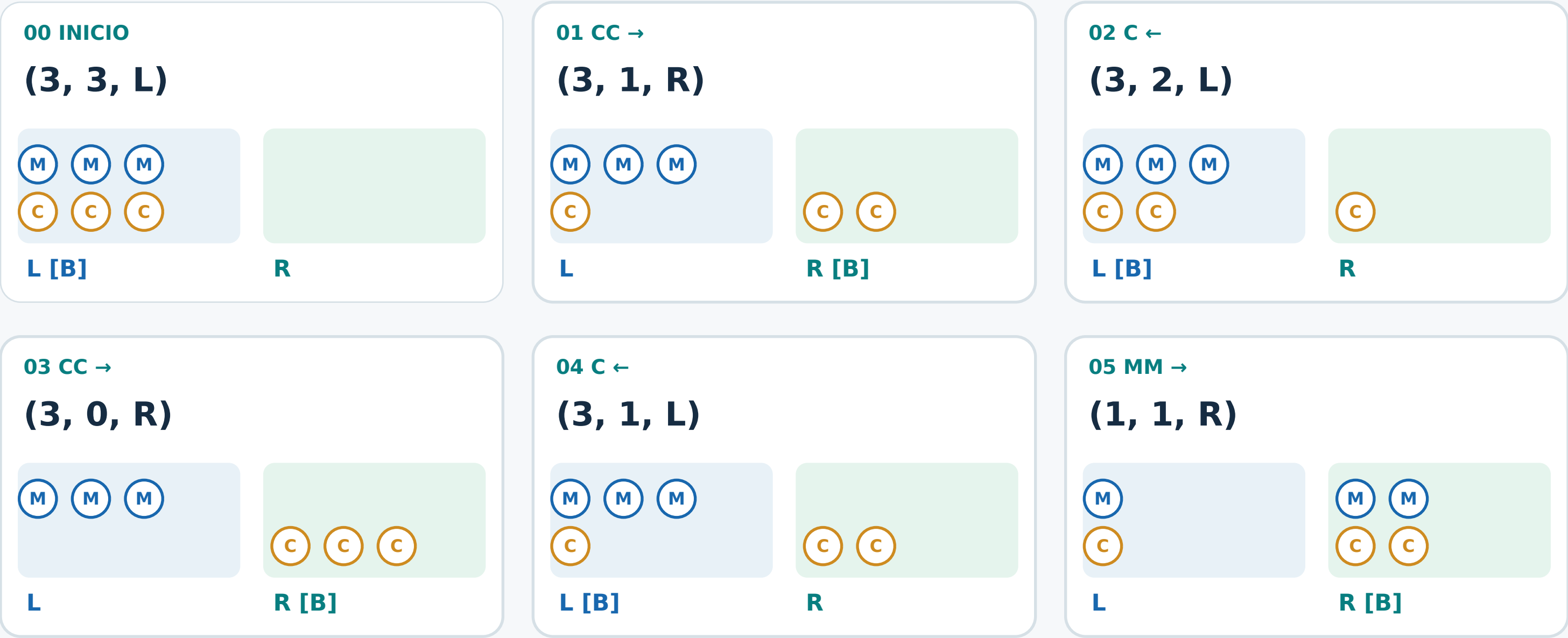
Filtro	Qué comprobamos	Ejemplo desde (3,3,L)
Capacidad	$1 \leq \text{pasajeros} \leq 2$	Barco vacío: descartado
Disponibilidad	Todos están en la orilla del barco	No se toman personas de la otra orilla
Seguridad en L	$m=0$ o $m \geq c$	M solo deja 2M y 3C: inválido
Seguridad en R	$3-m=0$ o $3-m \geq 3-c$	También se revisa la llegada
Coste	Una travesía cuesta 1	BFS minimiza cruces

Sucesores legales iniciales

$C \rightarrow (3,2,R) \cdot CC \rightarrow (3,1,R) \cdot MC \rightarrow (2,2,R)$. M y MM no cumplen la seguridad.

La solución en viñetas · 1/2

M = misionero · C = caníbal · [B] indica dónde queda el barco tras el cruce.

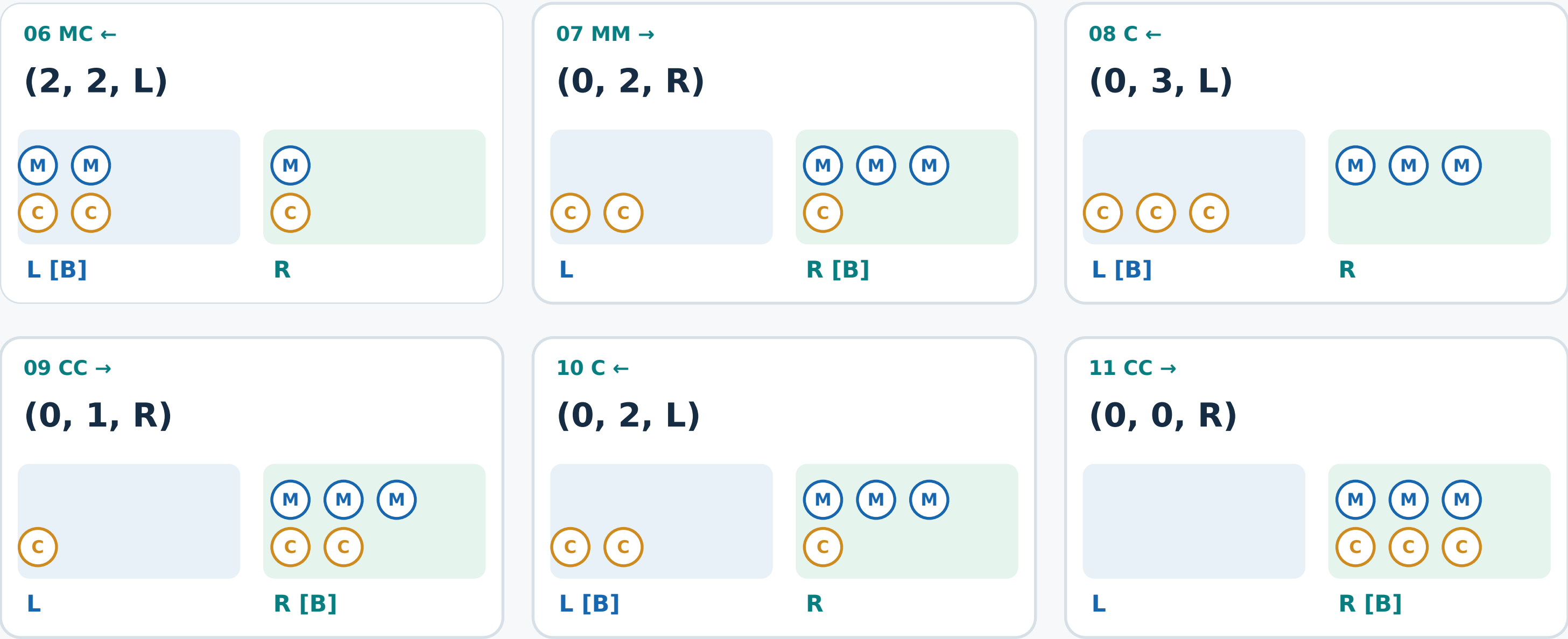


El cruce decisivo aún no llega

Después del paso 5 quedan 1M y 1C en L. Para continuar, debe regresar una pareja MC.

La solución en viñetas · 2/2

M = misionero · C = caníbal · [B] indica dónde queda el barco tras el cruce.

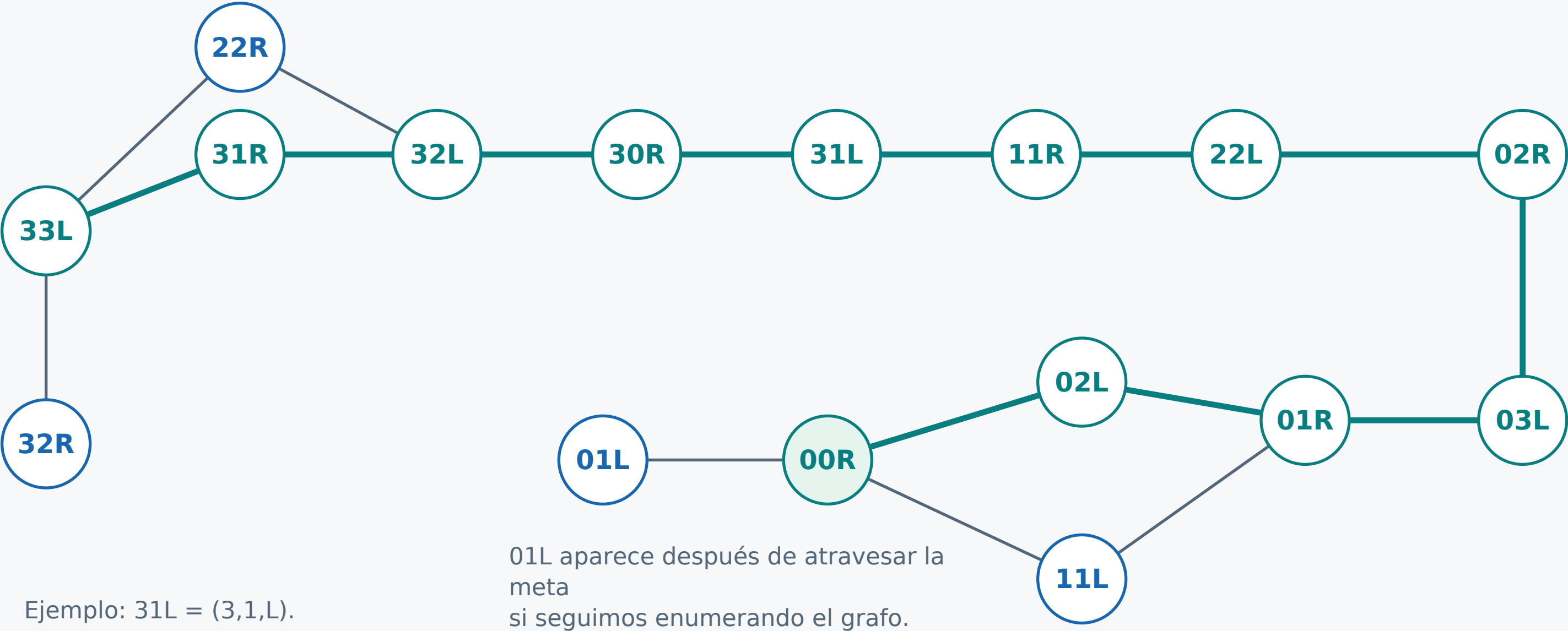


Meta alcanzada

Todos están en R. Se han realizado 11 cruces y nunca quedó una orilla con misioneros en minoría.

Contar estados no es contar caminos

Grafo completo alcanzable. Las conexiones son reversibles; el trazo verde marca una solución mínima.



32

combinaciones $4 \times 4 \times 2$

20

cumplen seguridad en las orillas

16

alcanzables desde el inicio

El barquero también forma parte del estado

La lámina usa (IslaA,IslaB), con los objetos L, O, C y B.

OBJETOS Y ACCIONES

L: lobo · **O**: oveja
C: col · **B**: barquero

El barquero cruza solo o con un objeto situado en su misma isla. Coste 1.

CONDICIÓN DE SEGURIDAD

Sin B, no pueden permanecer juntos:

L y O · el lobo y la oveja.
O y C · la oveja y la col.

Estas restricciones no están escritas en la imagen: aquí se hacen explícitas.

Inicio: (**{L,O,C,B}**, $\emptyset$)

Meta: ($\emptyset$, **{L,O,C,B}**)

Leer la imagen no sustituye validar sus aristas

El cruce de B solo desde el inicio que aparece en la lámina se descarta bajo estas reglas: deja L, O y C sin supervisión.

Siete cruces, ocho estados

Tras llevar la oveja, se transporta la col antes que el lobo; también existe la variante simétrica.

00 INICIO

ISLA A

L O C B

ISLA B

Ø

01 O →

ISLA A

L C

ISLA B

O B

02 Solo ←

ISLA A

L C B

ISLA B

O

03 C →

ISLA A

L

ISLA B

O C B

04 O ←

ISLA A

L O B

ISLA B

C

05 L →

ISLA A

O

ISLA B

L C B

06 Solo ←

ISLA A

O B

ISLA B

L C

07 O →

ISLA A

Ø

ISLA B

L O C B

O → · B ← · C → · O ← · L → · B ← · O →

Por qué hay que retroceder

Traer la oveja de vuelta no es un error: conserva la seguridad para el siguiente traslado.

DESPUÉS DE LLEVAR LA COL

Isla A: {L}
Isla B: {B,O,C}

Si B vuelve solo, deja O y C sin supervisión.

Debe traer O de vuelta.

DESPUÉS DE LLEVAR EL LOBO

Isla A: {O}
Isla B: {B,L,C}

L y C pueden permanecer juntos.

B puede volver solo.

7

coste óptimo: primera meta de BFS

10

estados seguros alcanzables con estas reglas

Formular no garantiza que exista solución

El primer partido AB ya está incluido en los contadores iniciales.

Estado = (partido, a, b, c)



(AB,1,1,0) → (AC,2,1,1)



(AB,1,1,0) → (BC,1,2,1)

SUCESOR

Se queda el ganador; entra quien esperaba.
Se incrementan los dos jugadores del nuevo partido.

META Y PODA

Meta: (a,b,c)=(10,15,17).
No generar un estado que supere alguno de esos totales. Coste 1 por transición.

Cada sucesor necesita su propia copia de los contadores.

La respuesta correcta es: no existe esa secuencia

Una prueba corta explica más que un árbol gigantesco sin meta.

42

participaciones: 10 + 15 + 17

21

partidos: 42 / 2

11

partidos mínimos de A, si empieza

A juega el primero y nunca puede descansar dos partidos seguidos.

Si A está fuera, entra en el siguiente. Por eso, en 21 partidos empezando dentro, necesita al menos 11 apariciones.



Patrón mínimo ilustrativo: 11 apariciones y 10 descansos.

La contradicción

Se piden solo 10 partidos de A. La enumeración de 562 estados acotados tampoco alcanza la meta: el máximo es 20 partidos.

Sibiu ya visitado... ¿pero a qué profundidad?

Límite $\ell=4$. Primero se explora Arad→Zerind→Oradea→Sibiu.



Sibiu en $d=3$: queda 1 acción dentro del límite.



Sibiu en $d=1$: quedan 3 acciones. La meta sí cabe.

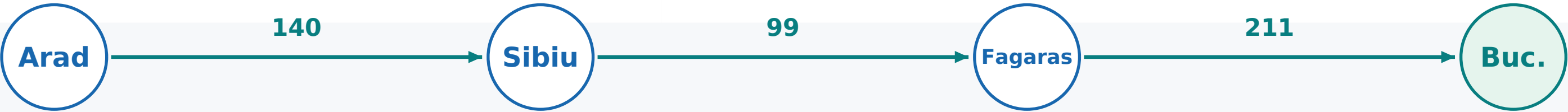
Error de la poda booleana

Si closed solo dice «Sibiu está», descarta la segunda llegada y puede declarar fallo aunque exista una solución de profundidad 3.

La reapertura conserva una oportunidad mejor

La prioridad –d organiza la frontera; la profundidad d decide si una llegada domina a otra.

Momento	Nodo extraído	Información en closed	Qué hacemos
Primera llegada	Sibiu, d=3	No estaba	Expandir; guardar d=3
Corte de la rama	Hijos con d=4	No se expanden	Aplicar el límite
Segunda llegada	Sibiu, d=1	Sibiu: 3	1<3: REABRIR
Siguiente	Fagaras, d=2	No estaba	Expandir
Meta	Bucharest, d=3	—	Terminar y reconstruir



3 acciones · coste 450 · DLS no garantiza el menor coste.

Tres salidas que no significan lo mismo

La meta se comprueba antes de rechazar un nodo situado justo en el límite.

SOLUCIÓN

Hay una ruta legal a una meta dentro de ℓ .

CUTOFF

El límite impidió seguir. No demuestra imposibilidad.

FALLO

Se agotaron las posibilidades sin que el límite ocultara una solución.

```

extraer n
si meta(n.estado): devolver solución
si n.d == límite: registrar cutoff; continuar
si estado no está en closed o n.d < closed[estado]:
  closed[estado] = n.d
generar y ordenar los sucesores
  
```

Regla para IDS

Aumentar ℓ cuando el resultado sea cutoff y reiniciar el control de visitados en cada iteración.

Un grafo, cinco formas de explorar

Instancia del ZIP: inicio 0, objetivo 9. No es el grafo 0→7 de Unit2.



Una ruta de coste 16

Estrategia	Ruta obtenida	d	Coste
BFS	0 → 5 → 2 → 6 → 9	4	16
UCS	0 → 5 → 3 → 6 → 9	4	16
DLS	0 → 5 → 2 → 1 → 8 → 3 → 6 → 9	7	43
Greedy	0 → 5 → 2 → 6 → 9	4	16
A*	0 → 5 → 3 → 6 → 9	4	16

Cálculo con sucesores por estado ascendente y desempate por id. DLS usa $\ell=7$.

Aprender a leer los nodos del ZIP

Se conserva el formato de tres filas usado en los árboles originales.

11; 16

9 S

4; 16; 0

Primera fila: id; value o prioridad.

Segunda fila: estado y marca.

Tercera fila: profundidad; g; h.

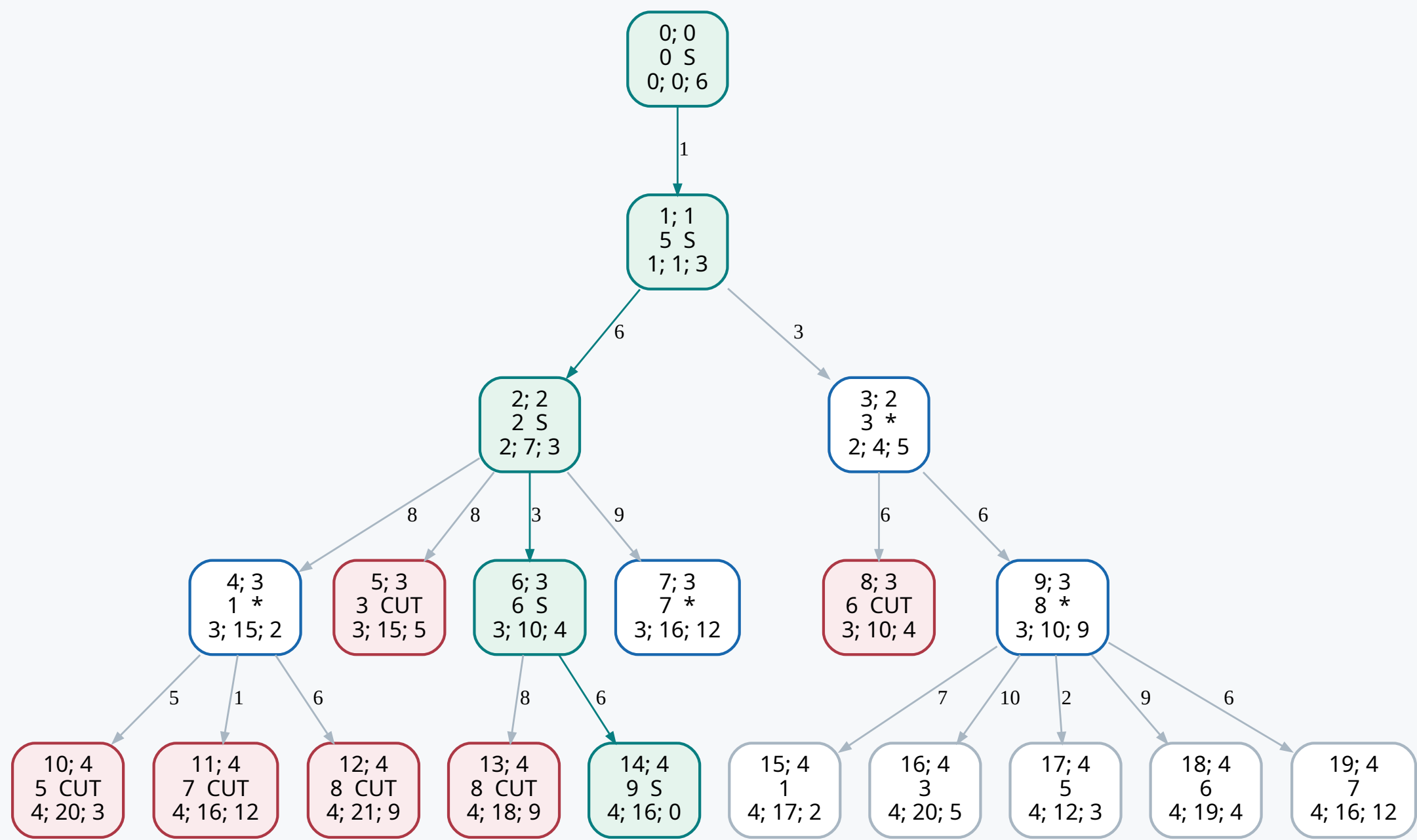
S	*	CUT	CUT_DEPTH
Ruta solución	Expandido, fuera de la ruta	Repetido descartado	Cortado por el límite

Prioridad según estrategia

BFS: d · DLS: −d · UCS: g · Greedy: h · A*: g+h. Nunca confundir el id con el estado.

Árbol completo · BFS

Verde: ruta solución · azul: expandido · rojo: descartado · gris: pendiente al terminar.

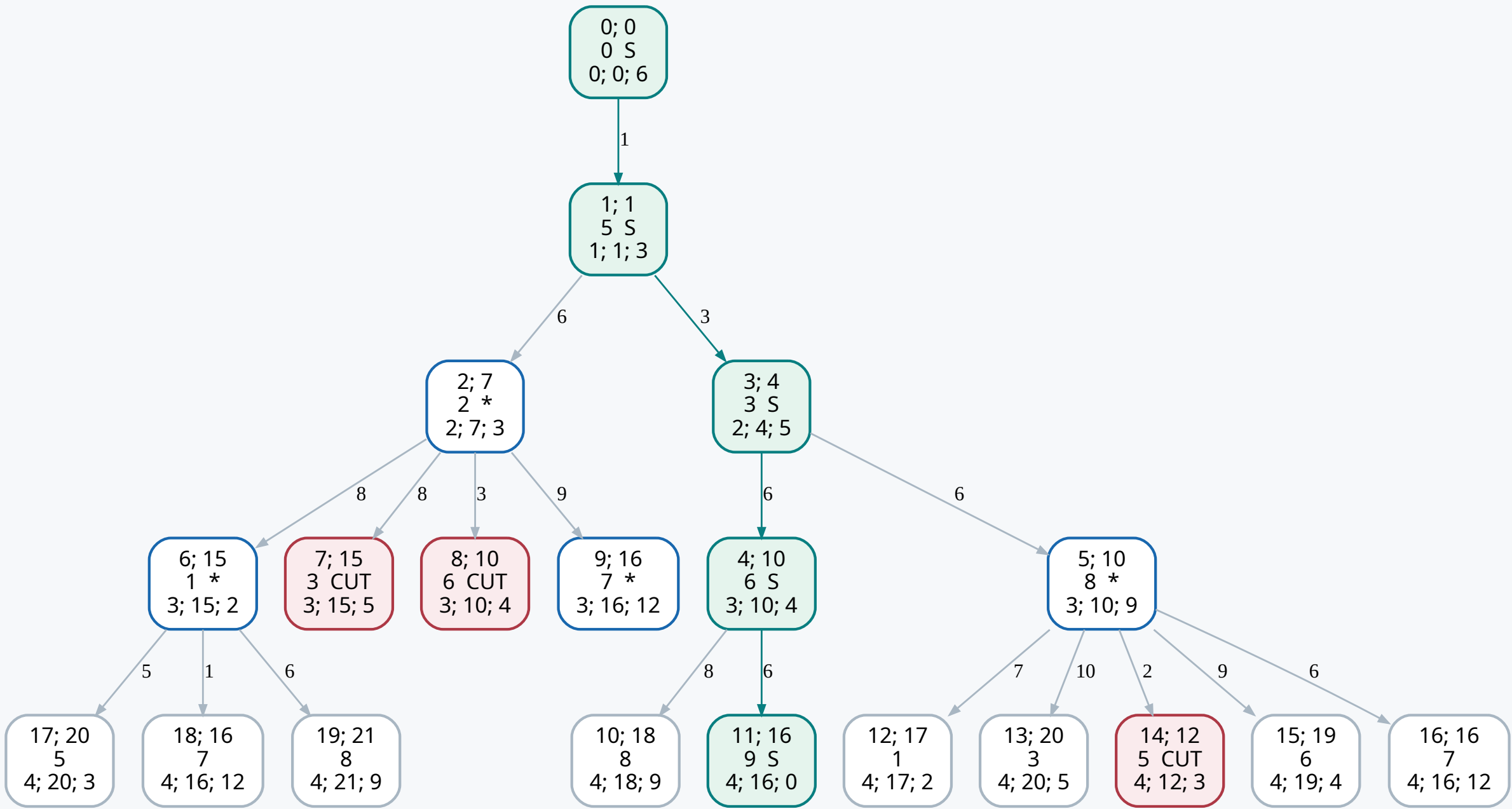


Ruta: 0 → 5 → 2 → 6 → 9 | Coste: 16

15 extracciones · 8 expansiones · 20 nodos creados, incluida la raíz.

Árbol completo · UCS

Verde: ruta solución · azul: expandido · rojo: descartado · gris: pendiente al terminar.

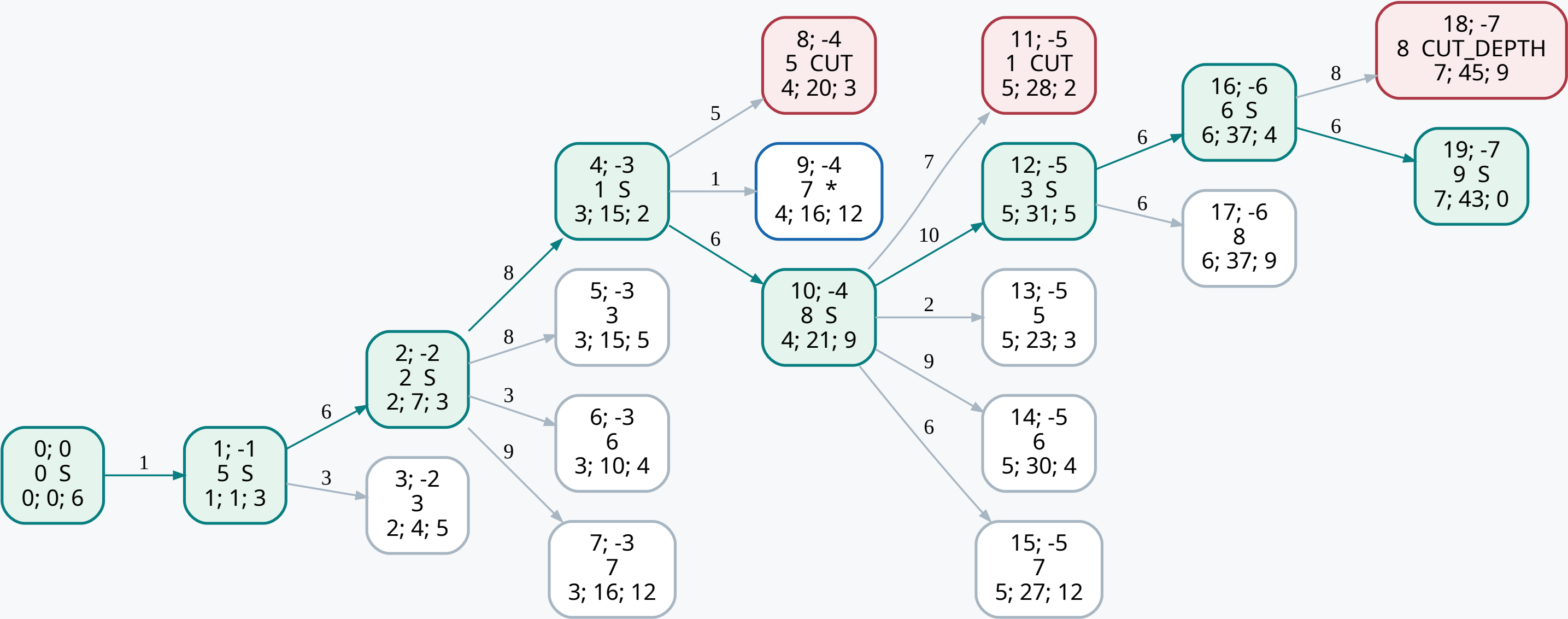


Ruta: 0 → 5 → 3 → 6 → 9 | Coste: 16

12 extracciones · 8 expansiones · 20 nodos creados, incluida la raíz.

Árbol completo · DLS con $\ell=7$

Verde: ruta solución · azul: expandido · rojo: descartado · gris: pendiente al terminar.

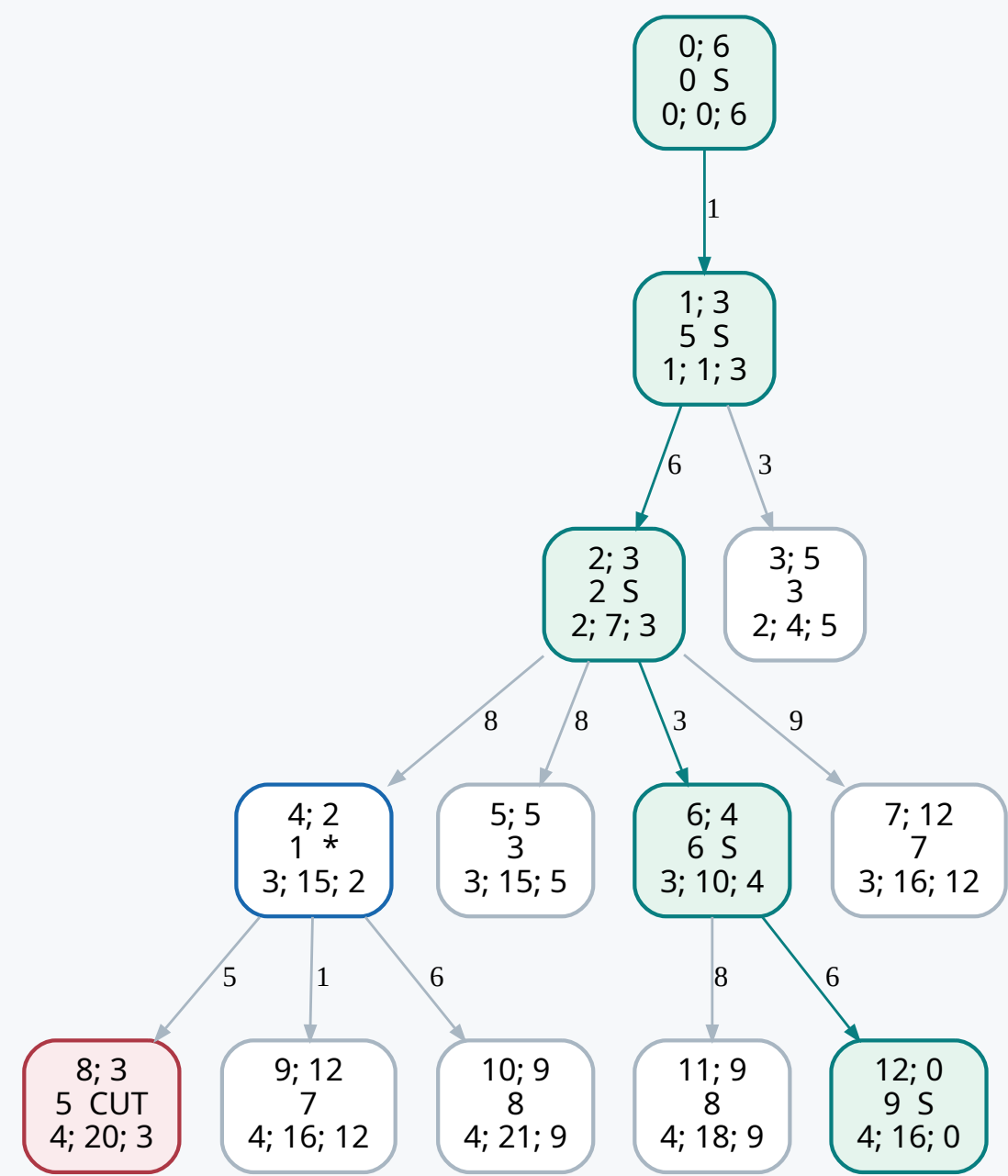


Ruta: 0 → 5 → 2 → 1 → 8 → 3 → 6 → 9 | Coste: 43

12 extracciones · 8 expansiones · 20 nodos creados, incluida la raíz.

Árbol completo · Greedy

Verde: ruta solución · azul: expandido · rojo: descartado · gris: pendiente al terminar.

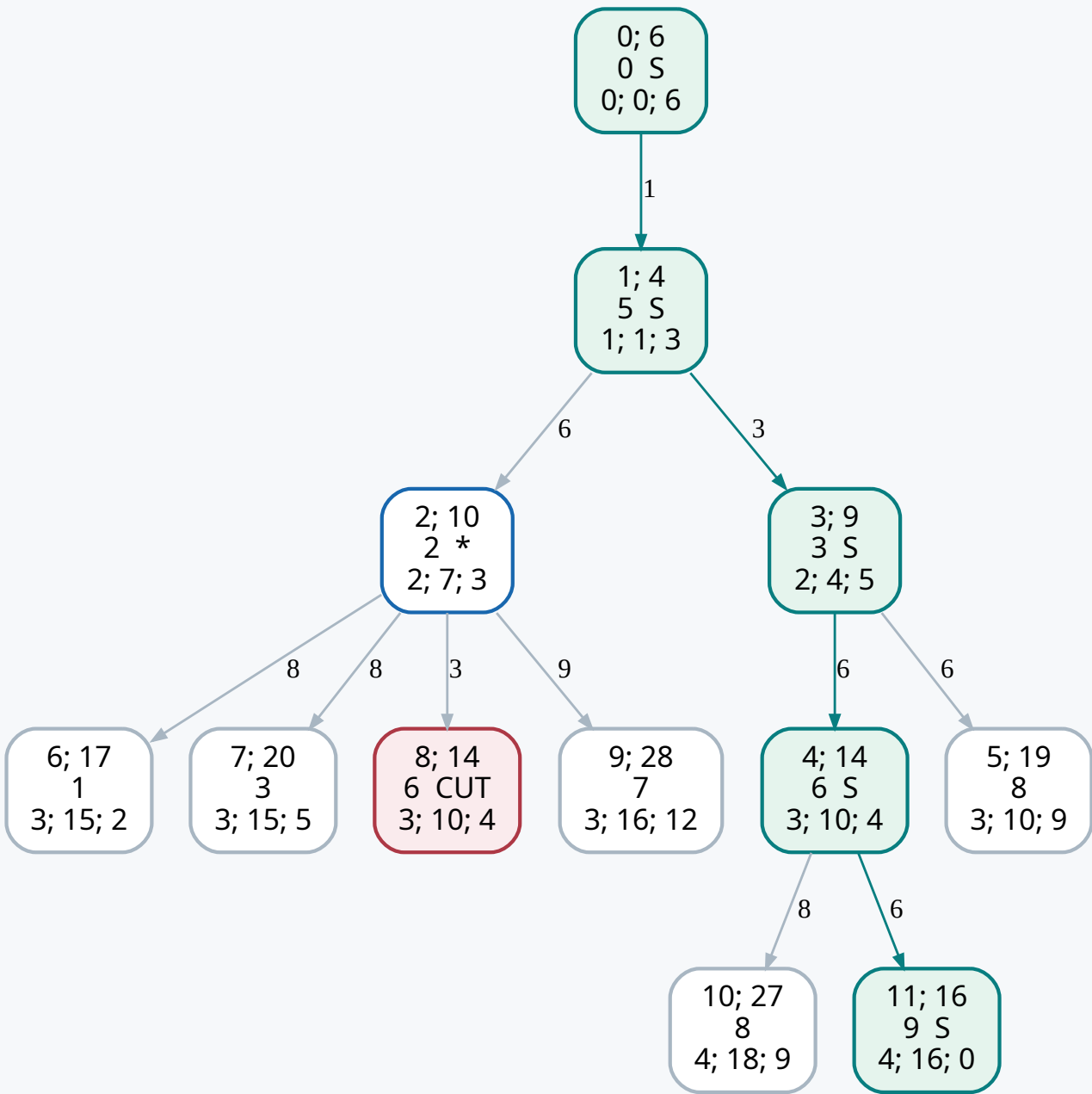


Ruta: 0 → 5 → 2 → 6 → 9 | Coste: 16

7 extracciones · 5 expansiones · 13 nodos creados, incluida la raíz.

Árbol completo · A*

Verde: ruta solución · azul: expandido · rojo: descartado · gris: pendiente al terminar.



Ruta: 0 → 5 → 3 → 6 → 9 | Coste: 16

7 extracciones · 5 expansiones · 12 nodos creados, incluida la raíz.

Pocos pasos no significa poco coste

Instancia: inicio azul 0; meta roja 10. Se explicita esta lectura de la lámina sin enunciado adicional.



BFS: 5 acciones · coste 24



UCS y A*: 6 acciones · coste 23

La diferencia está al final

4→10 cuesta 5. La alternativa 4→7→10 cuesta 2+2=4, aunque utiliza un paso adicional.

A* distingue coste real y estimación

Con $h(s)$ tomado de la etiqueta del grafo y $f=g+h$, los valores no son siempre crecientes.

id	Estado	g	h	f	Decisión
0	0	0	23	23	EXPANDE
1	6	4	9	13	EXPANDE
3	3	12	7	19	EXPANDE
2	2	6	14	20	EXPANDE
7	9	12	4	16	EXPANDE
6	3	11	7	18	REABRE
11	8	15	6	21	EXPANDE
5	8	16	6	22	CUT
9	4	19	3	22	EXPANDE
17	7	21	2	23	EXPANDE
24	10	23	0	23	META

Estado 3: primero llega con $g=12$; después con $g=11$. Se reabre la llegada mejor.

Una heurística admisible puede ser inconsistente

La admisibilidad compara con el coste real restante. La consistencia compara cada arista.

ADMISIBILIDAD

$h(s) \leq h^*(s)$

En el inicio: $h(0)=23$ y $h^*(0)=23$.

CONSISTENCIA

$h(s) \leq c(s,t)+h(t)$

En $0 \rightarrow 6$: $23 > 4+9$. No se cumple.

Estado	0	2	4	6	7	9	10
h	23	14	3	9	2	4	0
h*	23	17	4	19	2	11	0

Consecuencia práctica

Este A* permite reabrir estados si baja g. No se debe deducir la consistencia solo porque la heurística parezca razonable.

Auditar el grafo antes de buscar

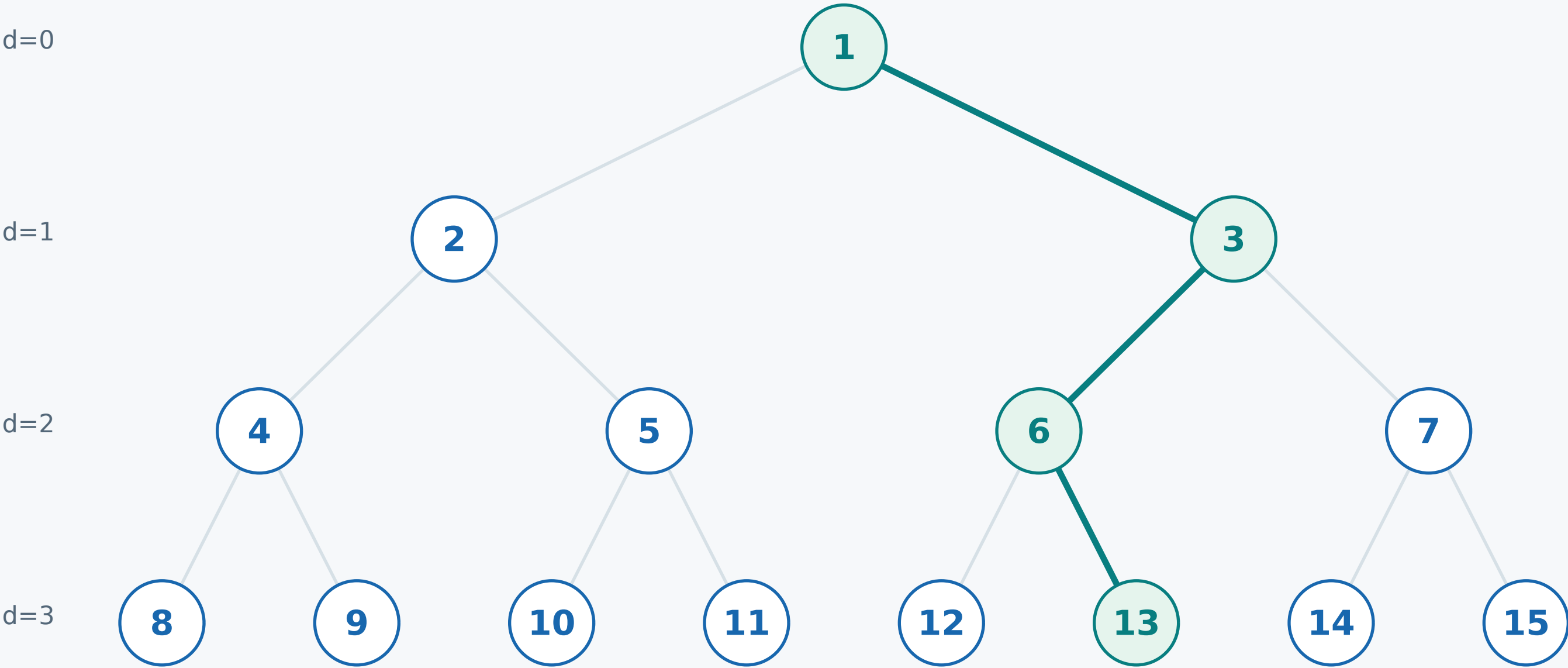
Lista de adyacencia transcrita: destino(coste). Es la referencia de cálculo de este dossier.

Estado	h	Sucesores dirigidos
0	23	6(4)
1	13	3(9)
2	14	3(5), 9(6)
3	7	0(4), 8(4)
4	3	0(8), 1(5), 3(8), 5(1), 7(2), 8(1), 9(10), 10(5)
5	15	0(10), 1(10)
6	9	2(2), 3(8)
7	2	1(6), 3(4), 4(3), 10(2)
8	6	1(2)
9	4	1(7), 4(7)
10	0	2(2)

BFS y Greedy devuelven 24 en esta instancia; UCS y A* con reapertura devuelven 23.

Llegar a 13: ver toda la estructura

Cada estado n genera $2n$ y $2n+1$. Raíz con profundidad 0.



Solución: 1 → 3 → 6 → 13 · Tres acciones

BFS, DLS e IDS visitan en órdenes distintos

Visitado significa aquí extraído/comprobado; se incluye la meta y se detiene allí.

BFS

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13

DLS · $\ell=3$

1, 2, 4, 8, 9, 5, 10, 11, 3, 6, 12, 13

IDS	Orden en cada reinicio
$\ell=0$	1
$\ell=1$	1, 2, 3
$\ell=2$	1, 2, 4, 5, 3, 6, 7
$\ell=3$	1, 2, 4, 8, 9, 5, 10, 11, 3, 6, 12, 13

Hacia atrás: 13 → 6 → 3 → 1, porque padre(n)=[n/2]. Ramificación inversa: 1.

A* desde Oradea hasta Bucharest

Conservamos el inicio de hw1soln; no lo cambiamos por el Arad del ejemplo del libro.



$$151 + 80 + 97 + 101 = 429$$

COSTE ACUMULADO g

0 → 151 → 231 → 328 → 429.

Es el coste recorrido, no una estimación.

HEURÍSTICA h

380 → 253 → 193 → 100 → 0.

Distancia en línea recta a Bucharest.

En la 3.ª edición, la tabla heurística es la figura 3.22; la hoja antigua remite a una figura 4.1.

No detenerse con la primera meta generada

Bucharest entra con coste 461, pero Pitesti permite mejorarla a 429.

Expandido	Frontera ordenada por f	Qué observar
Inicio	Oradea: 380	$g=0$, $h=380$
Oradea	Sibiu: 404 · Zerind: 445	Sibiu = $151+253$
Sibiu	Rimnicu: 424 · Fagaras: 426 · Zerind: 445 · Arad: 657	Rimnicu = $231+193$
Rimnicu	Fagaras: 426 · Pitesti: 428 · Zerind: 445 · Craiova: 537 · Arad: 657	Pitesti = $328+100$
Fagaras	Pitesti: 428 · Zerind: 445 · Bucharest: 461 · Craiova: 537 · Arad: 657	La meta no es la primera
Pitesti	Bucharest: 429 · Zerind: 445 · Craiova: 537 · Arad: 657	Actualizar 461 → 429
Bucharest	META	Se extrae con el menor f

Al cambiar el inicio, cambia el resultado

El ejemplo de Arad sirve para comparar cantidad de pasos y distancia total.

DESDE ARAD · 3 PASOS

Arad → Sibiu → Fagaras → Bucharest.

$$140 + 99 + 211 = 450.$$

Solución poco profunda. Es la ruta de DLS reparado y de BFS con el orden declarado.

DESDE ARAD · 4 PASOS

Arad → Sibiu → Rimnicu → Pitesti → Bucharest.

$$140 + 80 + 97 + 101 = 418.$$

UCS minimiza kilómetros, no saltos.

Pausa sobre Unit2, página 7

En ese dibujo no aparece Bucharest conectado: tomando solo ese grafo, no hay camino representado hasta la meta. En el mapa completo de la página 6 sí existe.

Un estado físico y un estado de creencias

La incertidumbre cambia lo que debe almacenar la búsqueda.

ESTADO FÍSICO

(posición, suciedadA, suciedadB).
 $2 \times 2 \times 2 = \mathbf{8 \text{ estados.}}$
Meta: ambas celdas limpias.

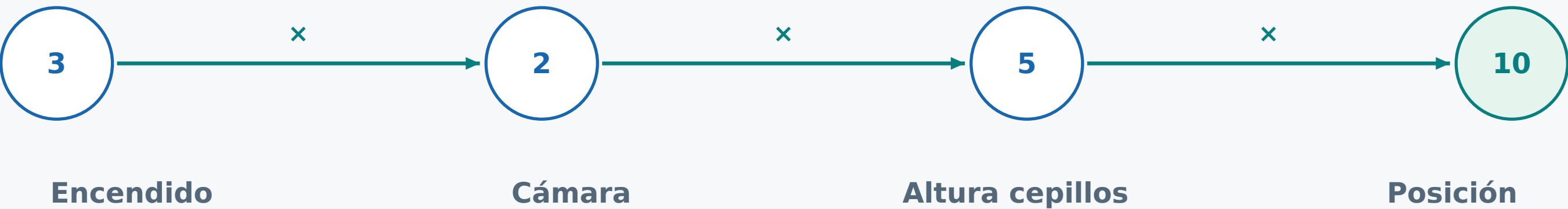
SIN SENSORES

El agente mantiene un conjunto de estados posibles. Una acción se aplica a todos los miembros del conjunto.

Acción	Conjunto posible después	Qué se sabe
Inicio	{1,2,3,4,5,6,7,8}	No se conoce la configuración
Right	{2,4,6,8}	Está en B
Suck	{4,8}	B está limpia
Left	{3,7}	Está en A y B está limpia
Suck	{7}	Ambas están limpias

¿300 estados o 307.200?

Depende de qué variables se incluyan en el modelo; el enunciado no lo detalla por completo.



300

$3 \times 2 \times 5 \times 10$: configuración del robot

307.200

300×2^{10} : si cada una de 10 celdas está limpia/sucia

Declarar el alcance del recuento

Los 300 casos no incluyen un mapa variable de suciedad. Los 307.200 requieren añadir diez variables binarias e independencia.

Transportar personas y carga

El peso total en cada cruce no puede superar 100 kg.

90 kg

Alex

80 kg

Brook

60 kg

Chris

40 kg

Dusty

20 kg

Carga

ESTADO

(A,B,C,D,E,b), cada valor indica una orilla.

E representa la carga, b el barco.

Inicio: todo en la primera orilla.

Meta: todo en la segunda.

ACCIÓN LEGAL

Un subconjunto situado con el barco, con al menos un explorador.

La suma de sus pesos es ≤ 100 .

Todos los seleccionados y el barco cambian de orilla.

Dos combinaciones especialmente útiles

Chris + Dusty = 100 kg. Brook + carga = 100 kg. Alex debe cruzar solo.

Una solución óptima de nueve cruces

La carga cruza con Brook; Chris y Dusty permiten recuperar el barco.

Cruce	Dirección	Transportados	Peso
1	→	Chris + Dusty	100
2	←	Chris	60
3	→	Alex	90
4	←	Dusty	40
5	→	Chris + Dusty	100
6	←	Chris	60
7	→	Brook + Carga	100
8	←	Dusty	40
9	→	Chris + Dusty	100

Capacidad respetada en todos los cruces. La primera meta de BFS aparece en profundidad 9.

Resolver la meta concreta de Unit2

El hueco se desplaza arriba, abajo, izquierda o derecha, sin salir del tablero.

7	2	4
5	.	6
8	3	1

INICIO

Búsqueda



1	2	3
4	5	6
7	8	.

META

FORMULACIÓN

Estado: nueve posiciones, 0=hueco.
Coste: 1 por movimiento.
Meta: (1,2,3,4,5,6,7,8,0).

HEURÍSTICA

Manhattan: sumar las distancias de las ocho fichas a su lugar final, sin contar el hueco.

Veinte movimientos verificados · 1/3

La letra de cada viñeta indica el movimiento del hueco que acaba de realizarse.

00 · INICIO

7	2	4
5	.	6
8	3	1

01 · D

7	2	4
5	3	6
8	.	1

02 · R

7	2	4
5	3	6
8	1	.

03 · U

7	2	4
5	3	.
8	1	6

04 · L

7	2	4
5	.	3
8	1	6

05 · L

7	2	4
.	5	3
8	1	6

06 · U

.	2	4
7	5	3
8	1	6

Secuencia: D R U L L U R R D L D L U R U L D R R D

Veinte movimientos verificados · 2/3

La letra de cada viñeta indica el movimiento del hueco que acaba de realizarse.

07 · R

2	.	4
7	5	3
8	1	6

08 · R

2	4	.
7	5	3
8	1	6

09 · D

2	4	3
7	5	.
8	1	6

10 · L

2	4	3
7	.	5
8	1	6

11 · D

2	4	3
7	1	5
8	.	6

12 · L

2	4	3
7	1	5
.	8	6

13 · U

2	4	3
.	1	5
7	8	6

Secuencia: D R U L L U R R D L D L U R U L D R R D

Veinte movimientos verificados · 3/3

La letra de cada viñeta indica el movimiento del hueco que acaba de realizarse.

14 · R

2	4	3
1	.	5
7	8	6

15 · U

2	.	3
1	4	5
7	8	6

16 · L

.	2	3
1	4	5
7	8	6

17 · D

1	2	3
.	4	5
7	8	6

18 · R

1	2	3
4	.	5
7	8	6

19 · R

1	2	3
4	5	.
7	8	6

20 · D

1	2	3
4	5	6
7	8	.

META
Coste 20

Secuencia: D R U L L U R R D L D L U R U L D R R D

Salto que eliminan grupos del mismo color

Se usa el tablero 4x4 que aparece en hw1soln; 0 significa casilla vacía.

1	2	2	1
2	1	3	2
3	1	3	2
.	2	3	.

MOVIMIENTO

Una bola salta una o más bolas contiguas de **un único color distinto al suyo**.

Aterriza en el hueco inmediatamente posterior. Se eliminan todas las bolas saltadas.

META Y COSTE

Queda exactamente una bola, en cualquier casilla. Coste 1 por salto, aunque elimine varias bolas.

Coordenadas (fila,columna), empezando en 1.

El ejemplo resuelto · 1/2

Los números identifican colores. El color visual es solo una ayuda y no cambia la regla.

00 · INICIO

1	2	2	1
2	1	3	2
3	1	3	2
.	2	3	.

01 · (2,1) → (4,1)

1	2	2	1
.	1	3	2
.	1	3	2
2	2	3	.

02 · (1,4) → (4,4)

1	2	2	.
.	1	3	.
.	1	3	.
2	2	3	1

03 · (1,1) → (1,4)

.	.	.	1
.	1	3	.
.	1	3	.
2	2	3	1

04 · (4,2) → (1,2)

.	2	.	1
.	.	3	.
.	.	3	.
2	.	3	1

El ejemplo resuelto · 2/2

Los números identifican colores. El color visual es solo una ayuda y no cambia la regla.

05 · (4,4) → (4,2)

.	2	.	1
.	.	3	.
.	.	3	.
2	1	.	.

06 · (4,1) → (4,3)

.	2	.	1
.	.	3	.
.	.	3	.
.	.	2	.

07 · (4,3) → (1,3)

.	2	2	1
.	.	.	.
.	.	.	.
.	.	.	.

08 · (1,4) → (1,1)

1	.	.	.
.	.	.	.
.	.	.	.
.	.	.	.

Una bola.
Ocho saltos.

Dos heurísticas seguras y un límite de evidencia

No todas las expresiones de la hoja pueden usarse sin revisar sus condiciones.

$h_0(s)=0$

Nunca sobreestima un coste no negativo.

A* con esta heurística equivale a UCS; con coste unitario obtiene el mismo coste mínimo que BFS.

$h_1(s)=\text{colores}(s)-1$

Un salto elimina bolas de un solo color; por tanto, no puede hacer desaparecer más de un color.

Para pasar de k colores a 1 hacen falta al menos $k-1$ saltos.

Recuento 7×7 : con k colores, $(k+1)^{49}$ configuraciones. El 7^{49} de la hoja presupone seis colores más el vacío.

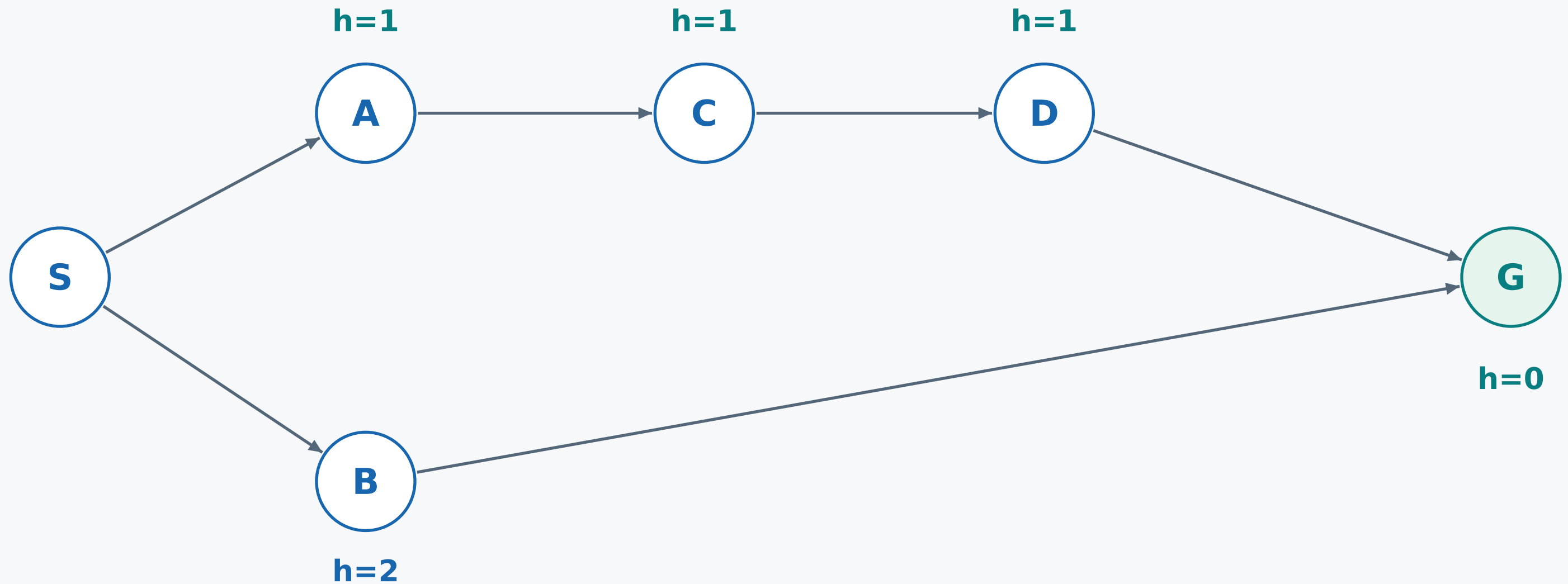
Precaución: $\min(\text{filas ocupadas}, \text{columnas ocupadas})$ vale 1 en una meta, así que esa expresión sin ajustar no es admisible.

Qué está comprobado y qué no

El ejemplo 4×4 tiene solución mínima de 8 saltos. No se adjuntaron testExample1/2/3.txt: no se inventan sus tiempos de ejecución.

Una mala guía puede explorar más que BFS

«Best-first» es una familia. Aquí interpretamos el ejercicio como Greedy, que prioriza h .



Greedy: S,A,C,D,G · 5 extracciones · coste 4.

BFS: S,A,B,C,G · 5 extracciones · coste 2.

Precisar qué se está contando

En el dibujo anterior hay igual número de extracciones, pero distinto coste. Lo ampliamos para obtener «peor» en ambos sentidos.



Rama larga: 5 acciones; $h(A_i)=1$. Rama corta: $S \rightarrow B \rightarrow G$, con $h(B)=2$ y $h(G)=0$.

GREEDY

Extrae S, A_1, A_2, A_3, A_4, G .

6 extracciones · 5 acciones.

BFS

Extrae S, A_1, B, A_2, G .

5 extracciones · 2 acciones.

Todos los arcos cuestan 1; A_1 antes que B ; meta al extraer. La heurística es deliberadamente no admisible: $h(B)=2$, aunque $B \rightarrow G$ cuesta 1.

Casos límite de los algoritmos locales

Las equivalencias de la hoja dependen también de la regla de aceptación y de parada.

Caso	Respuesta de la hoja	Matiz necesario
Local beam, $k=1$	Hill climbing	Equivalencia habitual si se acepta la mejor mejora y se comparte la regla de parada.
Beam sin límite de estados	BFS	Parte de una raíz y conserva todos los sucesores de cada nivel.
Annealing, $T \rightarrow 0^+$, sin parada	First-choice hill climbing	Se rechazan empeoramientos; las mesetas requieren definir el tratamiento de empates.
Genético, $N=1$	Paseo aleatorio por mutaciones	Cruzar un individuo consigo mismo no combina diversidad; no hay garantía de mejora sin elitismo.
$T=0$ no se sustituye literalmente en una división por T. El caso se interpreta como límite, tal como sugiere la hoja.		

Qué debe poder explicar el alumno

Una respuesta completa no es solo una lista de estados.

FORMULACIÓN

Qué contiene el estado; qué datos se conservan; qué acción es legal; cuál es la meta y cómo se calcula el coste.

TRAZA

Qué nodo sale de la frontera; qué prioridad lo decide; qué sucesores se generan; qué llegadas se descartan.

JUSTIFICACIÓN

Por qué cada paso es válido; cómo se reconstruye la ruta; qué propiedad permite afirmar optimalidad.

LÍMITES

Qué depende de un supuesto; cuándo solo hay cutoff; cuándo hay una prueba de imposibilidad.

Formular → Simular → Reconstruir → Verificar → Explicar

Qué se conserva y qué se ha completado

El libro fundamenta los algoritmos; los enunciados fijan cada instancia.

Material	Uso en este dossier
AIMA, 3. ^a ed.	Cap. 3: problema, búsqueda y heurísticas. Cap. 4: búsqueda local.
Unit2 / Unit1 / CourseIntroduction	Notación y estrategia docente; casos de aspiradora, exploradores y puzzle.
MissionariesAndCannibals / CurrencyExchange	Enunciados y formalización; completar sucesores y desarrollar las soluciones.
ExampleDLSPProblemPruning	Repetidos a menor profundidad; la hoja usa –d como prioridad.
Grafo2.zip / grafoclase	Instancias distintas: 0→9 y 0→10, respectivamente. Aristas transcritas.
Pinpong / espacioestadIslas	Ping-pong: prueba propia de imposibilidad. Islas: seguridad explicitada.
hw1soln	Objetivo 13, A* desde Oradea, búsqueda local y ejemplo 4×4 de Column Jump.

No se incluyen los PDF originales ni el libro en el paquete generado. Las figuras del dossier se han elaborado para esta explicación.

Cómo utilizarlo en el aula

Propuesta didáctica; no modifica la evaluación ni el calendario oficial de la asignatura.

Momento	Dinámica breve
Antes del algoritmo	Pedir a dos estudiantes que propongan una representación de estado.
Durante la traza	Ocultar la siguiente página; votar qué nodo se extrae a continuación.
Después de una ruta	Revisar coste, restricciones y por qué podría existir otra mejor.
Cuando no hay meta	Contrastar el ping-pong con el cutoff de DLS: son conclusiones distintas.
Al cerrar	Comparar BFS/UCS/A* sin cambiar el grafo, la meta ni el desempate.

Idea que conecta todos los ejercicios

Una búsqueda solo es tan correcta como el modelo que recorre y las reglas con las que decide descartar caminos.